

# TD7: Ingeniería de Datos

Password hashing

# Password hashing

— — —

Las contraseñas son información especialmente sensible dado que pueden permitir identificarse en más de un sistema:

- Necesitamos recibirlas de parte del **usuario** al crear su cuenta pero también en cada autenticación
- Necesitamos poder **validarla** contra nuestros registros cada vez
- No queremos que ningún **actor externo** pueda conseguirlas mediante un ataque
- No queremos que ningún **actor interno**, con acceso a los datos, pueda conseguirlas

# Password hashing

La técnica más común consiste en oscurecerlas mediante una función de hashing:

- Es una función **determinística**: a partir de un mismo texto obtengo siempre un mismo hash.
- Es una función **no reversible**: a partir del hash, hay múltiples posibles textos planos.
- Es **computacionalmente inviable** obtener el texto original a partir del hash: aunque teóricamente podría intentarse, en la práctica requeriría demasiado tiempo o recursos.

# Password hashing

---

Esto me permite:

- Computar y almacenar el hash al momento de crear la contraseña.
- Computar el hash en cada autenticación y validarlo contra la versión almacenada.
- Garantizar que ante una filtración de información o el acceso de un superusuario, no está en riesgo la privacidad de esa contraseña.

# Password hashing - Rainbow tables

— — —

El problema es que esto no las hace inmunes a ataques de **rainbow tables**:

- Listas de hashes **precomputados** (para contraseñas comunes).
- Permiten comparar muy rápidamente **inmensas cantidades** de hashes para encontrar matches
- Si bien no permite descryptar todas las contraseñas, ataca una **porción muy grande**:

➤ [https://en.wikipedia.org/wiki/2012\\_LinkedIn\\_hack](https://en.wikipedia.org/wiki/2012_LinkedIn_hack)

# Password hashing

— — —

<https://defuse.ca/checksums.htm>

Para hashear un texto con distintos algoritmos

<https://crackstation.net/>

Crackeador de contraseñas hasheadas

# Password hashing - Salting



— — —

Una técnica para dificultar el uso de rainbow tables es el **salting**:

- Por cada password a almacenar, se genera un string al azar (**salt**)
- Se almacena este string junto con la información de la cuenta
- Cada vez que tiene que calcular el hash (tanto al crear la password como al validarla) se **concatena el nuevo string** con la password original.

# Password hashing - Salting



Hashing simple:

| usuario    | password_md5                     |
|------------|----------------------------------|
| jgervasoni | 2df19e32f25968c72ecee90e9d058c0f |
| fbugni     | e96682fce047bd09327f03f564a94f74 |
| fpeitti    | 2e7d610470967d980461a17c674cd394 |

Texto a hashear

passjulio

1234

mimascota2018



# Password hashing - Salting



— — —

Hashing con salt:

| usuario    | salt    | password_md5                     |
|------------|---------|----------------------------------|
| igervasoni | CqiKKuh | d0cd15cd1b718f89946637418772534f |
| ichiapella | f9fGUAR | 81dc9bdb52d04dc20036dbd8313ed055 |
| ppiccoli   | OXqs7f1 | cd51fb257e42e97630edb75f6cfe2a70 |

Texto a hashear

passjulioCqiKKuh

1234f9fGUAR

mimascota2018DfvWoKW

**Cierre**



# Bibliografía

— — —

-  **[ELM16] Fundamentals of Database Systems, 7th Edition, R. Elmasri, S. Navathe, 2016.**
-  Database Security: Capítulo 30